

Then we saw a shift towards SOA (service oriented architecture), which broke the applications into modules but this too had its flaws. The aggregation layer was supposed to be a thin one but, in reality, it got heavy with the transformations and the logic that went into it. It created another level of coupling between the internal and external components of the system. And then came microservices. But before we explore it further, there are certain trade-offs with microservices as well, which we need to keep in mind when recommending this architectural style to solve business problems.

What is a microservice?

A microservice is not a new concept. In fact, many of the microservices concepts are similar to SOA patterns. Microservices are about breaking up a complex system into more manageable, smaller units of work that can be developed and deployed independently. They are about breaking complex system level problems into smaller problems so that they become easier to solve.

Understanding the core concepts and components of a microservice

Services are at the core of the microservices based architecture. There are no hard and fast rules on the sizing of microservices. They should neither be too fine grained, which will make them difficult to manage, nor should they be too coarse grained, which will require them to have cross-domain operation. Making them too fine grained will also increase the distribution, leading to more latency in service calls.

Services should be built around the domain and not on a data object or a business object. They should expose domain-specific CRUD operations. It may so happen that a single service serves multiple business processes and data objects based on the use case. The concept will be clearer if we relate our understanding of a class in the OOP paradigm. A class is developed to work on one specific type of object, and it provides the operations to perform activities on those objects. It follows the single responsibility principle (SRP). Similarly, microservices operate on a single domain. There should not be any cross-domain operations within a service.

It is recommended that communications between services are over HTTP, using ReST based services. The beauty of this communication approach is that it allows coding in any language that supports ReSTful services. This helps drive agility in development as developers can choose languages that they are comfortable with. The ReST based communication is also known as protocol-aware heterogeneous interoperability, which means that the services are bound to a protocol (HTTP in this case) and communicate in a heterogeneous environment, using this protocol.

In addition to communication, there is also the need for collaboration among services to perform a certain business process. The question is whether that collaboration needs to be synchronous or asynchronous. The best way to reduce

latency in a distributed environment is to depend on an event based asynchronous communication model. Asynchronous communication can be achieved through message queues like AMQ, RabbitMQ or through distributed streaming platforms like Kafka. However, it is important to put some checks and balances when leveraging asynchronous communication. This will require us to build a monitoring system that will match the view of the business process. In reality, however, it may need to be a mix and match of asynchronous and synchronous communication, based on the requirements of a particular use case.

Logging and tracking are the most critical components of a distributed architecture to monitor the service call chains and for aggregating logs related to those call chains. In the distributed microservices architecture, it is important to determine the flow of events through the entire system. Tracing solutions like Spring Cloud Sleuth help to monitor those flows, which then can be exported to Zipkin for visualisation. Sleuth does this tracing by enhancing the log to add unique IDs (the trace ID) to each request that enters the application and a Span ID to each step of the request. A single trace comprises of multiple spans, which identify a specific step or section in the request. Spring Cloud has out-of-the-box support from Sleuth and Zipkin.

The API layer is another core component of the microservices architecture. This layer is an abstraction on top of the services, and acts as an aggregated proxy for all the services. It hides the implementation details of the services from the outside world. The API layer hides the location and the IP address of the service from the client. For the client, the name of the service will never change. The API layer takes the service name, enquires the service discovery component (like Eureka or Consul), and returns the actual IP address and port where the service is running. It also helps in versioning the services.

The complexity of a distributed architecture like microservices cannot be managed without automation. It is therefore very important to embrace DevOps principles and practices while developing microservices architecture based applications. The code integration and deployment should be fully automated, and an immediate feedback mechanism must be built into this automation.

The conclusion is just the beginning

There is a shift happening in how we develop applications and the microservices architecture will be at the core of this shift. The applications of the future will be developed leveraging this architecture. Organisations have already started breaking up their monoliths into microservices and this trend is only going to grow further in the near future. 

By: Rajib Deb

The author leads the open source practice for a multi-national company. He can be contacted at rajib76@yahoo.com.